Übungsblatt 3: Speicherzugriff und Kontrollfluss

Papieraufgaben, Rechenwege und Venus-Vergleich

Ziel und Arbeitsweise

Sie berechnen Speicheradressen, verfolgen veränderte Register und Byte und begründen Sprungpfade sowie Schleifengrenzen. Die Inhalte von Vorlesung 1 und 2 bilden das Vorwissen; die neuen Befehle werden in Vorlesung 3 erklärt. Eine höhere Programmiersprache wird nicht vorausgesetzt.

Bearbeiten Sie Aufgaben 1–6 zunächst ohne Simulator. Die **60 Punkte** dienen der Selbstkontrolle und sind keine Prüfungsbewertung. Erst Aufgabe 7 verlangt den Vergleich mit der Kursdistribution von Venus. Die optionale Vertiefung ist zusätzliche Reserve.

Aufgabe	Schwerpunkt	Minuten	Punkte
1	Lesen, rechnen, speichern	9	10
2	Effektive Adresse und Ausrichtung	8	8
3	Teilzugriffe und Bytezustände	9	10
4	Store-Format codieren/decodieren	9	10
5	Entscheidungen und Sprungpfade	10	12
6	Schleifen-Trace und leerer Fall	9	10
7	Array-Summe in Venus	30	—

Planungsannahme für vier Schulstunden: 76 Minuten Vorlesung einschließlich Rückfragen, 54 Minuten Papierarbeit, 30 Minuten Venus, zehn Minuten Quiz und zehn Minuten Auswertung. Bei höherem Unterstützungsbedarf kann Aufgabe 4 nachbesprochen werden; die Vertiefung liegt außerhalb dieser 180 Minuten.

Gemeinsame Annahmen

RV32I ist die RISC-V-Integerbasis mit 32-Bit-Registern. Wir verwenden keine komprimierte Erweiterung C: Jeder native Befehl belegt vier Byte. Adressen zählen Byte. Daten werden Little Endian abgelegt, das niederwertige Byte zuerst. Registerarithmetik speichert die unteren 32 Bit; `x0` liefert null und verwirft Schreibversuche. In Speicheraufgaben werden nur ausdrücklich angegebene Regionen und natürlich ausgerichtete Zugriffe benutzt.

Aliasnamen: `t0=x5`, `t1=x6`, `t2=x7`, `t3=x28`, `t4=x29`, `t5=x30`, `t6=x31`, `s0=x8`, `s1=x9`, `a0=x10`, `a1=x11`. `PC` bedeutet Program Counter, Programmzähler. Nicht genannte Register- oder Speicherinhalte sind unbekannt; jede Aufgabe hat ihren eigenen Anfangszustand.

Aufgabe 1: Lesen, rechnen, speichern

10 Punkte · 9 Minuten · Papier

Eine gültige, beschreibbare Region ab `0x1000` enthält die fünf Wörter $A = [10, 20, 30, 40, 50]$. `s1=0x1000` bleibt zunächst die Basis. Es werden nacheinander ausgeführt:

```
lw   t0, 8(s1)
addi t1, t0, 12
sw   t1, 12(s1)
```

a) Notieren Sie nach jeder Zeile die effektive Adresse, soweit ein Speicherzugriff stattfindet, den neuen Registerinhalt und die veränderten Speicherbyte. Ein Strich bedeutet „keine Änderung“ beziehungsweise „kein Zugriff“ (je 2 Punkte).

Zeile	effektive Adresse	Registeränderung	Speicheränderung
1			
2			
3			

b) Schreiben Sie das vollständige Array nach Zeile 3 auf. Welches Quellregister liefert beim Store die Daten, welches die Adresse? (2 Punkte).

c) Welche vier Byte liegen danach an den Adressen `0x100C` bis `0x100F`? Geben Sie außerdem den Endwert von `s1` an (2 Punkte).

Rechnungen und Antworten b)–c):

Aufgabe 2: Ausrichtung ist eine Adressfrage

8 Punkte · 8 Minuten · Papier

Nur die Byteadressen `0x1000` bis einschließlich `0x1013` seien hier zugreifbar. Bestimmen Sie die effektive Adresse und entscheiden Sie unabhängig über natürliche Ausrichtung und gültigen gesamten Bytebereich (je Zeile 1 Punkt).

Basis	Offset	Breite	Adresse	ausgerichtet?	Bereich gültig?
0x1001	+3	Wort			
0x1001	+2	Halbwort			
0x1002	+2	Wort			
0x1003	-3	Wort			
0x1003	0	Byte			
0x1014	0	Wort			

Die Breiten sind Wort = 4 Byte, Halbwort = 2 Byte, Byte = 1 Byte.

b) Warum genügt ein gerader Offset nicht, um einen Halbwortzugriff auszurichten? Muss ein unausgerichteter Zugriff auf jeder RISC-V-Implementierung zwingend mit einer Ausnahme abbrechen? (2 Punkte).

Begründungen:

Aufgabe 3: Teilzugriffe vollständig verfolgen

10 Punkte · 9 Minuten · Papier

Es gilt `s1=0x2000`. Die gültigen Byte sind anfangs:

Adresse	`0x2000`	`0x2001`	`0x2002`	`0x2003`
Inhalt	`80`	`FF`	`00`	`80`

a) Diese vier Loads finden vor jeder Speicheränderung statt. Bestimmen Sie jeweils das achtstellige Registerwort und den Zahlenwert entsprechend der Erweiterungsart (je 1 Punkt).

Instruktion	Registerwort	Zahlenwert
lb t0, 0(s1)		
lbu t1, 0(s1)		
lh t2, 2(s1)		
lhu t3, 0(s1)		

b) Anschließend ist `t4=0x11223344` gegeben. Es werden `sb t4, 1(s1)` und danach `sh t4, 2(s1)` ausgeführt. Geben Sie die vier Speicherbyte nach jedem Store und das abschließende 32-Bit-Wort ab `0x2000` an (4 Punkte).

c) Bleibt `t4` unverändert? Warum ist für diese Wirkung kein eigenes `sbu` nötig? (2 Punkte).

Bytezustände und Begründungen:

Aufgabe 4: S-Format von Hand

10 Punkte · 9 Minuten · Papier

Referenz: Bits 31:25 = `imm[11:5]`, 24:20 = `rs2`, 19:15 = `rs1`, 14:12 = `funct3`, 11:7 = `imm[4:0]`, 6:0 = `opcode`. Für `sw`: funct3 `010`, opcode `0100011`. Registerfelder haben fünf Bit; `t0=x5`, `t1=x6`, `s1=x9`.

a) Codieren Sie `sw t1, -8(s1)`. Zeigen Sie das zwölf Bit breite Offsetmuster, seine beiden Teile, alle sechs Felder und das Hexwort (6 Punkte).

b) Decodieren Sie `0x0054A623`. Rekonstruieren Sie den Offset und schreiben Sie den Befehl mit Aliasnamen. Welches Registerfeld fehlt diesem Format gegenüber dem Registerformat? (4 Punkte).

Zwölf-Bit-Offset a) und Aufteilung:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode

Wort und Hexdarstellung a), Rückrechnung b):

Aufgabe 5: Welche Instruktionen laufen wirklich?

12 Punkte · 10 Minuten · Papier

Der erste Befehl liegt bei `0x1000`. Alle gezeigten Befehle sind **nativ**. `jal x0, EndIf` ist hier die feste Entwicklung von `j EndIf` und verwirft den Linkwert durch `x0`.

```
    bne  s0, s1, ElseBlock
    addi t0, x0, 1
    jal  x0, EndIf
ElseBlock:
    addi t0, x0, 2
EndIf:
    addi t1, x0, 9
```

a) Geben Sie die Adressen der fünf Instruktionen und der beiden Labels an (3 Punkte).

b) Notieren Sie für `s0=s1=7` und für `s0=7, s1=8` die ausgeführten Befehlsadressen in Reihenfolge, die Adresse nach dem letzten Befehl und die Endwerte `t0` und `t1` (4 Punkte).

c) Welche signed Byteabstände werden beim Branch und beim Jump zum jeweiligen Ziel verwendet? Der Abstand zählt ab der Sprunginstruktion (2 Punkte).

d) Ergänzen Sie die genommene/nicht genommene Entscheidung für `blt` und `bltu` mit Quellen `0xFFFFFFFF` und `1` in dieser Reihenfolge (2 Punkte).

e) Eine Variante lässt den Jump zwischen den Blöcken weg. Welcher Fehler entsteht für gleiche Eingaben? Eine neue Adressrechnung ist nicht verlangt (1 Punkt).

Adressliste, Wege und Begründungen:

Aufgabe 6: Eine Schleife einschließlich Endtest

10 Punkte · 9 Minuten · Papier

Ein gültiges Array mit genau drei Wörtern $A = [4, -1, 2]$ beginnt bei $B =$ `0x3000`. Anfang: `t0=0`, `t1=3`, `s0=0`, `s1=B`. `t2` ist zunächst unbekannt.

```
Loop:
    bge  t0, t1, Done
    lw   t2, 0(s1)
    add  s0, s0, t2
    addi s1, s1, 4
    addi t0, t0, 1
    j    Loop
Done:
```

a) Füllen Sie alle Zustände **vor** `bge` aus, einschließlich des Tests, der beendet (4 Punkte).

Test Nr.	t0	s1	s0	Branch genommen?
1				
2				
3				
4				

b) Wie viele Loads und Tests werden ausgeführt? Welches Wort und welchen signed Wert hat `t2` am Ende? (2 Punkte).

c) Setzen Sie nur die Anzahl auf `t1=0` und beginnen Sie neu. Wie lauten Summe, Pointer und Anzahl der Loads? Muss `t2` dafür vorher null sein? (2 Punkte).

d) Warum sind ein Pointerupdate um eins oder ein Limit vier für dieses Array unpassend? Unterscheiden Sie Ausrichtung und Objektgrenze (2 Punkte).

Weitere Antworten:

Aufgabe 7: Array-Summe in Venus

30 Minuten · Rechner und Papierprotokoll

Implementieren Sie die Summe der ersten N Werte aus `.word 10, 20, 30, 40, 50`. Vorbedingung: $0 \leq N \leq 5$. Die Eingabedaten bleiben unverändert.

1. Legen Sie die Daten unter `myArray:` im Datenabschnitt an. Nutzen Sie einen Codeabschnitt mit `.globl main` und Startlabel `main:`.
2. Laden Sie die Basis mit `la s1, myArray`. Initialisieren Sie `t0=0`, `t1=N` und `s0=0`.
3. Verwenden Sie das Schleifenmuster aus Aufgabe 6. Der Test liegt vor `lw`, der Pointer wächst um vier Byte, der Zähler um eins.
4. Am Label `Ergebnis_pruefen:` vor der Ausgabe prüfen Sie `s0`, `t0` und `s1`. Notieren Sie vorher die erwarteten Werte.
5. Ausgabe und Ende in Kurs-Venus: `mv a1, s0`, `li a0, 1`, `ecall`, danach `li a0, 10` und `ecall`. Die Dienstnummer ist kein Summenwert.
6. Prüfen Sie $N = 5$, $N = 0$, $N = 1$ und danach den Datensatz $[-3, 8, -10, 1]$ mit $N = 4$. Nach jeder Änderung neu assemblieren und starten. Protokollieren Sie Ergebnis, Pointerabstand zur tatsächlichen Basis und Load-Anzahl.

Fall	Vorhersage	Beobachtung	erste Abweichung
N=5			
N=0			
N=1			
gemischte Werte			

Eigener Code:

Optionale Vertiefung: Teilzugriffe vergleichen

Zusätzliche Reserve · nicht Teil der 84 Übungsminuten

Öffnen Sie vl3_teilzugriffe.s. Der Prüfpunkt liegt vor dem Abschlussdienst. Sagen Sie zunächst die Werte von `t0` bis `t6` und die beiden Wörter bei `target` voraus. Vergleichen Sie dann mit den Registern und der Speicheransicht. Klären Sie, ob die Ansicht einzelne Byte oder zusammengesetzte Wörter zeigt.

Öffnen Sie anschließend vl3_if_else.s. Prüfen Sie gleiche und ungleiche Eingaben. Notieren Sie die tatsächlich ausgeführten Instruktionen, nicht nur die letzte Ergebniszahl. Erst nach der Vorhersage können Sie in einer Kopie den Jump entfernen und die veränderte Folge erklären.

Vorhersage und Vergleich: